Orderflow Engine — Recorrido de la solución

Jesús Ariel González Bonilla

2026-10-17

Pregunta que responde este documento: ¿cómo se resolvió la prueba, paso a paso, y qué decisiones de ingeniería se tomaron en el camino?

Este recorrido está escrito para quien evalúa la prueba. Sigue el trabajo en el orden en que ocurrió: leer el enunciado, escribir la especificación, planificar, construir el motor de precios y la confirmación del pedido, demostrarlos y entregar. Cada cifra sale de un archivo de los repositorios o del comando que se indica a su lado. El estado verificado vive en un solo lugar, orderflow-engine-docs/handoff/HANDOFF.md §3, y este documento lo cita.

1. El enunciado en una página

El enunciado (orderflow-engine-docs/assignment/) describe un marketplace B2B donde los tenderos les compran a distribuidores de bebidas. Una sola plataforma multi-tenant atiende varios países (Colombia, Ecuador, Guatemala, Perú) y varias marcas. Un pedido pasa por catálogo, carrito, checkout (la cotización), confirmación e historial. Cinco palancas comerciales actúan sobre el precio y pueden coexistir en un mismo pedido: descuento por escala, combo, obsequio, impuestos y cupo de crédito. El plazo es de cinco días, y el lenguaje es Go, Java (Spring Boot o WebFlux) o NestJS.

Parte	Qué pide	Dónde lo responde este documento
A1. Motor de cálculo	Un endpoint o caso de uso que reciba el carrito y devuelva el desglose por línea y los totales: subtotal bruto, descuentos con su origen, obsequios con su cantidad, base gravable, impuestos y total. Las palancas coexisten, y su orden debe definirse y justificarse. Un pedido a crédito debe indicar si cabe en el cupo disponible	§5
A1. Núcleo no funcional	El total cotizado y el confirmado deben ser idénticos, aunque cambien precios o palancas entre un momento y otro. Nada de punto flotante para dinero, con tipo, escala decimal, regla de redondeo y puntos de redondeo explícitos. Moneda, escala y tasas no pueden quedar quemadas en el código	§5, §6.1
A1. Golden tests	Seis casos obligatorios: descuento en dos tramos; combo cuyos productos no aparecen como descuento suelto; obsequio con su cantidad; combo con impuestos; pedido a crédito que excede el cupo; cotizar y confirmar dan el mismo total	§5.5

A2. Confirmación del pedido	En tres releases distintas se confirmó un pedido, se devolvió un número de orden y después el pedido no aparecía en el historial. POST /orders recibe una clave de idempotencia, para que los reintentos nunca dupliquen un pedido ni consuman el cupo dos veces. Un 201 significa que el pedido existe y se puede consultar. Los efectos hacia push y ERP nunca se pierden ni se duplican, también con el sistema externo caído. El modelo de consistencia y lo que sacrifica quedan documentados	§6
A3. ADR	Máximo una página: decisiones y alternativas descartadas, qué no se implementó a propósito, qué revisar en un code review y un riesgo con su mitigación	§7
Entrega	Un repositorio Git con el código y las pruebas, un comando único para correrlas, un README con cómo correrlo, el orden de las palancas, la política de redondeo y el modelo de consistencia, el ADR y un historial de commits legible	§11, §14
Front opcional	Una pantalla de resumen del pedido en Angular o Flutter, con una hora de trabajo como máximo. Tres defectos no deben ocurrir: textos cortados, controles tapados por la barra del sistema o por el teclado, e imágenes de un mismo grid escaladas con criterios distintos. Suma explicar cómo prevenirlos a nivel de componente	§8

El mismo enunciado llama a los requisitos no funcionales de A1 el corazón del ejercicio. El diseño empezó por ahí: una cotización sellada para que lo cotizado sea lo confirmado, un tipo decimal para el dinero con redondeo por línea, y datos por país en lugar de constantes.

2. Lectura del problema: ambigüedades y supuestos

Antes de escribir código, se leyó el enunciado frase por frase. Cada frase que admitía dos lecturas se convirtió en un supuesto con un ID estable en orderflow-engine-docs/spec/SPEC.md §13, y cada supuesto se hace cumplir con una regla de negocio (BR-xx) y una prueba.

ID	Ambigüedad del enunciado	Supuesto adoptado	Por qué
A-01	Las escalas son "acumulativas por tramos"	Tramos marginales: cada porcentaje se aplica solo a las unidades dentro de su tramo, y los	Los tramos se acumulan segmento por segmento, como los tramos de un impuesto. El golden test 1 fija esta lectura con cifras exactas

		porcentajes nunca se suman	
A-02	Un SKU puede tener a la vez una regla por SKU y una por familia	La regla por SKU saca ese SKU de las reglas por familia	Gana la regla específica, y ninguna unidad recibe dos descuentos por escala (TierPrecedenceTest)
A-03	El orden en que las unidades de una familia llenan los tramos	El orden normalizado por SKU	El mismo carrito siempre da el mismo precio, sin importar el orden de sus líneas (BR-00)
A-04	Si las unidades de combo y de obsequio cuentan para la escala	Quedan fuera de los umbrales y del descuento por escala	Sin doble beneficio: una unidad con precio de combo no se descuenta otra vez (golden test 2)
A-05	Cuánto dura vigente una cotización	15 minutos por defecto, definidos por país como dato	Acota cuánto tiempo se respeta un precio sellado, que es el riesgo del ADR
A-06	Quién llama: el enunciado no dice nada de autenticación	La autenticación queda fuera del alcance. Tenant y cliente llegan en los encabezados X-Tenant-Id y X-Customer-Id	Un gateway los inyectaría. En el cliente web, el selector de tienda hace las veces de inicio de sesión
A-07	Cómo aparece un combo en el desglose	Una sola entrada de descuento por combo. La asignación por línea solo sirve para derivar la base gravable de cada línea	El golden test 2 prohíbe mostrar los productos del combo como descuentos sueltos, y el impuesto igual necesita una base por línea
A-08	Si el cupo se reserva al cotizar	No. Al cotizar, la verificación es solo informativa	La verificación que obliga corre una sola vez, dentro de la transacción de confirmación. Si dos cotizaciones abiertas caben, ahí se resuelve: una se confirma y la otra recibe 409
A-09	Cancelación y liberación de cupo	Fuera del alcance	El enunciado no las pide
A-10	Qué significa "una condición de compra" para un obsequio	Una cantidad mínima pagada de un SKU o familia, o un subtotal bruto mínimo antes de descuentos. La línea de obsequio siempre	El obsequio lo dispara lo que se compra y nunca cambia lo que se paga por producto

		va aparte de la línea pagada	

Algunas preguntas el enunciado se las hace directamente al candidato. Cada respuesta es una regla de la especificación:

Pregunta del enunciado	Respuesta	Dónde
¿Qué tipo, escala y regla de redondeo para el dinero?	Money con un BigDecimal y una política de moneda; escala y modo de redondeo tomados de los datos del país; HALF_UP en los cuatro países sembrados	SPEC §4
¿Se redondea por línea, por total o por impuesto?	Solo por línea. Los totales son sumas de líneas ya redondeadas y nunca se vuelven a redondear	SPEC §4
¿Qué significa la implicación tributaria del obsequio?	Decide la política del país para obsequios: EXEMPT o TAXED sobre el valor de referencia del obsequio	SPEC BR-04
¿Qué garantiza un 201?	La fila del pedido queda confirmada en la base de datos y legible antes de serializar el 201 (invariante I-201)	SPEC §7.2
¿Qué modelo de consistencia, y qué se sacrifica?	Fuerte para pedido, cupo, clave y outbox en una sola base de datos; eventual para push y ERP	SPEC §7.5

3. Primero la especificación

La especificación se escribió y se integró antes de la primera línea de código del servicio. Los primeros pull requests del repositorio de documentación, el 2026-10-14, son la constitución y la especificación, luego el plan, las tareas y el contrato, y después el modelo de datos y la arquitectura. El código del servicio empezó el 2026-10-15.

3.1 La constitución

orderflow-engine-docs/spec/constitution.md reúne 15 principios no negociables. Cada uno dice cómo se verifica, porque un principio sin verificación es apenas un deseo. Los que más marcaron la solución:

Artículo	Principio	Verificado por
P-01	El dinero nunca es un número binario de punto flotante	DomainPurityArchTest, MoneyTest
P-02	El total cotizado es el total confirmado: el pedido referencia la cotización sellada y no guarda montos	QuoteConfirmRegressionIT (golden test 6)
P-03	El 201 se envía solo después del commit de la transacción que creó el pedido	ConfirmOrderCommitOrderingIT, OrderReadYourWritesIT
P-04	Todo efecto externo pasa por el outbox, escrito en la transacción del pedido	OutboxWriteIT, OutboxRelayIT

P-05	Ninguna llamada remota dentro de una transacción de base de datos	TransactionBoundaryArchTest
P-06	El dominio de precios es puro: sin Spring, JDBC, HTTP ni reloj	DomainPurityArchTest, los golden tests
P-07	Ningún dato de país en el código	GoldenPricingTest#multiCountry, datos semilla de Liquibase
P-08	Toda regla de negocio tiene una prueba que se vio fallar	El ciclo de cada tarea (§10), puntaje de PIT de al menos 80%
P-12	Ninguna afirmación sin evidencia	HANDOFF.md §3
P-15	Un alcance menor nunca baja el estándar: ni TODO, ni marcadores provisionales, ni código muerto	check-docs, revisión

Los demás artículos fijan la división de idiomas (inglés para ingeniería, español por defecto para los usuarios), los documentos de entrega bilingües, la documentación gobernada, los comentarios que explican el porqué y los README de cada repositorio, que responden cinco preguntas.

3.2 SPEC, reglas de negocio y criterios de aceptación

orderflow-engine-docs/spec/SPEC.md abre con cuatro garantías, cada una con su mecanismo:

#	Garantía	Mecanismo
G-1	El total cotizado es el total confirmado	Una cotización sellada, de solo inserción. El pedido la referencia y no guarda montos
G-2	Nada de punto flotante para dinero	Un value object con BigDecimal, escala y redondeo tomados de los datos del país, redondeo por línea
G-3	Un 201 significa que el pedido existe; los reintentos nunca lo duplican ni consumen el cupo dos veces	Pedido, cupo, clave de idempotencia y outbox en una sola transacción ACID, y la respuesta después del commit
G-4	Los efectos en push y ERP nunca se pierden ni se aplican dos veces	Outbox transaccional, relay con lease y reintentos, y consumidores que deduplican por event_id

Después vienen el alcance (§1, con una tabla explícita de lo que entra y lo que no), el glosario y las reglas de negocio:

Regla	Tema
BR-00	Normalización del carrito: unir SKU repetidos, ordenar por SKU, rechazar cantidades inválidas, carritos vacíos y SKU desconocidos
BR-01	El orden en que se aplican las palancas, con la razón de cada paso
BR-02	Descuento por escala: tramos marginales, redondeo una vez por línea, la regla por SKU por encima de la regla por familia
BR-03	Combos: aplicaciones, descuento, una entrada por combo, asignación por mayor residuo
BR-04	Obsequios: condición, línea de obsequio aparte, valor de referencia, política

	tributaria del país
BR-05	Impuesto por línea y las identidades de totales que todo desglose debe cumplir
BR-06	Cupo de crédito: informativo al cotizar, consumo condicional al confirmar
BR-07	Ciclo de vida de la cotización: OPEN, CONFIRMED, EXPIRED, derivado y nunca guardado

BR-02 y BR-03 traen ejemplos resueltos a mano antes de escribir código. Esos ejemplos se volvieron los fixtures de los golden tests 1 y 4, así que la especificación y las pruebas comparten las mismas cifras.

Los criterios de aceptación son 52 filas, de AC-01 a AC-53. AC-22 salió del alcance, y los ID nunca se renumeran. Cada fila nombra la regla a la que sirve, la tarea que la entrega y la clase de prueba que la demuestra.

3.3 Trazabilidad

SPEC §14 enlaza el enunciado con el código en tres tablas:

Tabla	Vínculo que mantiene
§14.1 Cobertura del enunciado	Cada requisito del enunciado (REQ-A1-1 … REQ-A1-6, REQ-A1-T1 … T6, REQ-A2-1 … 4, REQ-A3, REQ-D, REQ-F, REQ-P) con sus criterios de aceptación
§14.2 Criterios de aceptación	Cada criterio con su regla, su tarea y la prueba que lo verifica
§14.3 Más allá del enunciado	Todo lo que el enunciado no pidió, con su etiqueta (calidad, estabilidad, escalabilidad), su beneficio y su origen

La cadena siempre es la misma: viñeta del enunciado → REQ → AC → BR o sección de la SPEC → tarea → clase de prueba. Una revisión medida el 2026-10-15 encontró especificadas 21 de 21 viñetas del enunciado (cross_reference_check.py, registrado en el HANDOFF).

3.4 Gobierno de la documentación y su verificación automática

La documentación sigue reglas escritas (orderflow-engine-docs/governance/documentation-rules.md): una pregunta por documento, un mapa de documentos, front matter en cada documento, documentos de entrega en inglés y en español con los mismos encabezados, y documentos técnicos internos en inglés. Una regla que ninguna verificación hace cumplir se deteriora (trampa TR-03 del HANDOFF), por eso las reglas las hacen cumplir herramientas del repositorio orderflow-engine-tools:

Verificación	En qué falla
docs-check/check-docs.mjs	Un archivo Markdown fuera de las ubicaciones permitidas; un archivo que falta en el mapa de documentos; un par de idiomas incompleto o con distinta cantidad de encabezados; front matter ausente o inválido; un enlace relativo roto o que sale de su repositorio; un ADR de más de 500 palabras; un encabezado obligatorio ausente (por ejemplo, los cuatro

	temas del README de la API); bundles de i18n con claves distintas; una gráfica de avance que no coincide con la tabla de tareas del HANDOFF; una línea de commit en TASKS que rompe las convenciones; un Word o PDF generado cuya fuente cambió después de construirlo
integrity/check-integrity.mjs	Archivos de ignorados o .gitattributes faltantes, reglas de fin de línea, archivos de Docker, .env.example y secretos, en los seis repositorios
consistency/	Conteos, referencias cruzadas, idioma y paridad de pares entre documentos

4. El plan

orderflow-engine-docs/spec/TASKS.md convierte la especificación en tareas. También fija nueve restricciones globales (GC-1 … GC-9), el ciclo de cada tarea, un foco de revisión con nueve bordes riesgosos (RF-1 … RF-9), el plan por días y la línea de corte. orderflow-engine-docs/spec/plan.md lista cada decisión técnica con sus alternativas descartadas (DEC-01 … DEC-20) y el registro de riesgos (RISK-01 … RISK-07).

4.1 Tareas y fases

Tarea	Título	Fase
T-00	Especificación, gobierno, README, borrador del ADR	1. Preparación
T-01	Andamiaje de la API y comando de prueba del espacio de trabajo	2. Andamiaje
T-02	Dinero y política de moneda	3. Núcleo
T-03	Pipeline de precios y arnés de golden tests	3. Núcleo
T-04	Descuentos por escala	3. Núcleo
T-05	Combos y asignación	3. Núcleo
T-06	Obsequios y su política tributaria	3. Núcleo
T-07	Redondeo y multipaís	3. Núcleo
T-08	Esquema y datos semilla	3. Núcleo
T-09	Manejo de errores e i18n de la API	3. Núcleo
T-10	Caso de uso de cotización y endpoints	3. Núcleo
T-11	Confirmación del pedido	3. Núcleo
T-12	Invariantes bajo concurrencia y fallas	3. Núcleo
T-13	Historial de pedidos	3. Núcleo
T-14	Relay del outbox	3. Núcleo
T-15	Limpieza por retención	3. Núcleo
T-18	Conectar las palancas a las cotizaciones	3. Núcleo
T-16	Documentación final y verificación	3. Núcleo
T-17	Cliente web: resumen y flujo completo (opcional)	4. Web
T-19	Correcciones tras la vista previa de T-17	4. Web
T-20	Sembrar Guatemala y Perú	4. Web

Fase	Tareas	Cerrada el	Merge commit en `main`
1. Preparación	T-00	2026-10-14	chore: close preparation phase

			(T-00)
2. Andamiaje	T-01	2026-10-15	chore: close scaffold phase (T-01)
3. Núcleo	T-02 … T-16, T-18	2026-10-15	chore: close core phase (T-16)
4. Web	T-17, T-19, T-20	2026-10-17	chore: close web phase (T-17)

4.2 Plan por días y línea de corte

Día	Tareas planeadas
D1	T-00 … T-03
D2	T-04 … T-07
D3	T-08 … T-10
D4	T-11 … T-14
D5	T-15, T-18, T-16, luego T-17 por partes, luego T-19 y T-20

La línea de corte del MVP queda después de T-16. Todo lo que pide el enunciado está por encima de ella. T-17, el front opcional, queda por debajo y se dividió en partes temáticas que se pueden cortar sin riesgo: después de cada parte web el cliente compila, corre en Docker y pasa sus pruebas, así que detenerse después de cualquier parte no deja rutas, claves ni endpoints a medio conectar. El plazo de cinco días es el riesgo RISK-07 del plan, y el avance se iba a contrastar con el plan al final del día 4 (decisión abierta OD-02).

4.3 El plan frente a la realidad

El historial de merges del repositorio de la API muestra el orden que se siguió de verdad. El 2026-10-15 se integraron primero T-02 y T-03 (dinero y pipeline), luego T-08, T-09, T-14, T-15 y T-10 (esquema, errores, outbox, limpieza, cotizaciones), y después T-11, T-12 y T-13 (confirmación, sus invariantes, historial). Las palancas T-04 … T-07 vinieron a continuación, y luego T-18 y T-16. Es decir, el camino de persistencia y confirmación se construyó antes que las palancas. TASKS registra la consecuencia: T-10 se construyó antes que T-04 … T-06, así que su camino de cotización solo cargaba precios y tasas de impuesto. Por eso se agregó T-18, que conecta las palancas sembradas a las cotizaciones, con QuoteLeversIT reproduciendo los casos de escala, combo y obsequio a través de POST /api/v1/quotes, antes de que T-16 cerrara el núcleo.

Después de T-17 se agregaron dos tareas. T-19 reúne las correcciones que salieron de recorrer el stack completo: nombres de productos y clientes en el idioma de los tenants, un tope de 1,000,000 por cantidad, códigos de moneda junto a los montos, el idioma elegido recordado, reinicio por Docker y un paso de extremo a extremo en test-all. T-20 agrega Guatemala y Perú solo como datos.

5. El motor de precios (A1) paso a paso

5.1 El tipo de dinero

Aspecto	Decisión
Tipo	Money(BigDecimal amount, CurrencyPolicy policy), un value object inmutable. Una regla de ArchUnit prohíbe float y double en pricing.domain

Escala	Dato: la fila de la moneda (COP 0; USD, GTQ y PEN 2). Nunca java.util.Currency, que reporta COP con 2 decimales (trampa TR-01 del HANDOFF); otra regla de ArchUnit lo prohíbe en el dominio
Modo de redondeo	Dato: la fila del país. HALF_UP en los cuatro países sembrados
Almacenamiento	El dinero es NUMERIC(19,4) y las tasas NUMERIC(7,6). Un precio con más decimales de los que admite su moneda se rechaza al cargarlo (PriceScaleValidationIT)
Comparación	Siempre compareTo, nunca equals, porque equals también compara la escala
API	El dinero viaja como cadena JSON con la escala de la moneda, y el código ISO de moneda aparece una vez por cuerpo. El cliente web formatea esa cadena y nunca la convierte en número

5.2 Normalización y pipeline

El motor es una sola función pura: Breakdown PricingEngine.price(Cart, PricingSnapshot). Primero la capa de aplicación resuelve la foto de los datos: precios unitarios, palancas activas, tasas de impuesto y política del país en un mismo instante. El motor no ve base de datos, reloj ni framework, así que es determinista y los golden tests pueden fijar su resultado (P-06).

Antes de calcular, el carrito se normaliza (BR-00): se unen los SKU repetidos, las líneas se ordenan por SKU y la entrada inválida falla de inmediato con un código estable: una cantidad de cero o menos, no entera o mayor que 1,000,000 → 400 INVALID_QUANTITY; un carrito vacío → 400 CART_EMPTY; un SKU desconocido o de otro tenant → 422 UNKNOWN_SKU. Una cantidad de obsequio derivada por encima del mismo límite también es un 400, calculada con aritmética que detecta desbordamiento, así que nunca da la vuelta ni responde 500.

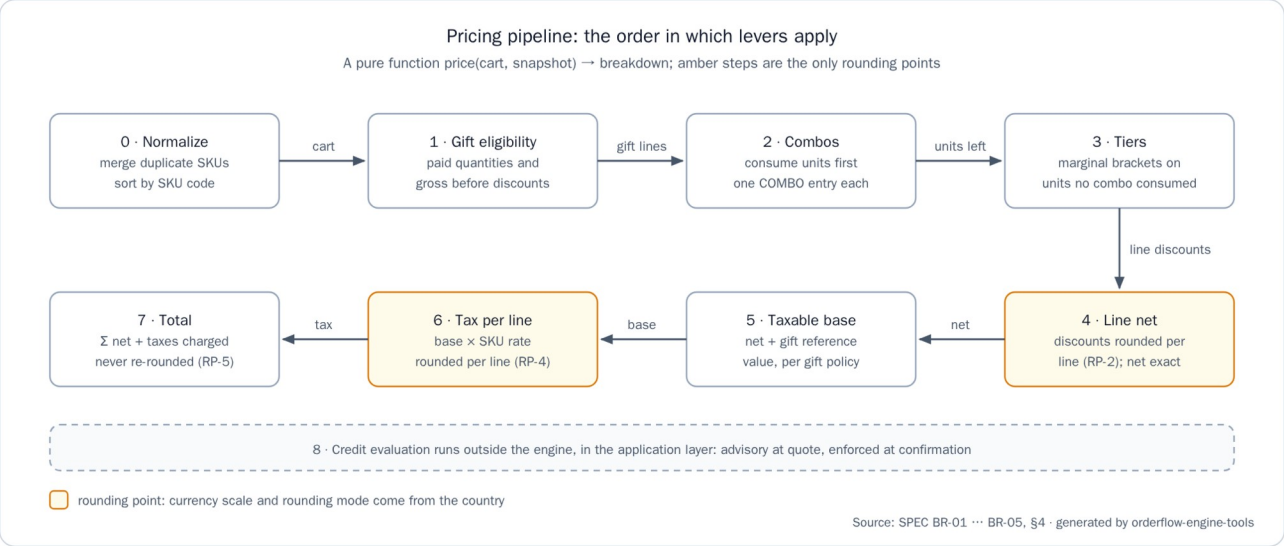


Figura 1. Pipeline de precios: el orden en que se aplican las palancas

5.3 Orden de las palancas y sus razones

Paso	Qué	Por qué aquí
1	Elegibilidad de obsequios, sobre	El obsequio lo dispara lo que el cliente

	cantidades pagadas y el subtotal bruto antes de descuentos	compra y nunca cambia lo que paga por producto
2	Combos	Un combo es un precio de paquete negociado. Consume sus unidades primero, para que no reciban además un descuento por escala
3	Descuentos por escala sobre las unidades que ningún combo consumió	El incentivo genérico por volumen se aplica solo a lo que el paquete no tarifó
4	Neto de la línea = bruto − componentes de descuento	Los descuentos comerciales se liquidan antes del impuesto
5	Base gravable	El enunciado exige el impuesto sobre la base después de los descuentos comerciales
6	Impuesto por línea	Cada SKU tiene su propia tasa
7	Total	Necesita todos los pasos anteriores
8	Evaluación del cupo, en la capa de aplicación	Necesita el total final y no es una palanca de precio, así que queda fuera del motor puro

Cada palanca tiene una regla precisa:

- **Escalas (BR-02).** Una regla apunta a un SKU o a una familia y define tramos {from, percent}. El descuento de una línea se calcula exacto y se redondea una vez por línea, nunca una vez por segmento de tramo (borde de revisión RF-1).
- **Combos (BR-03).** Aplicaciones = el piso del menor qty / minQty entre los SKU del combo. Descuento = valor de lista de las unidades consumidas − precio del combo × aplicaciones. Un combo que no bajaría el precio no consume nada. El descuento se reparte entre las líneas participantes en proporción a su valor bruto consumido por el método del mayor residuo, con empates resueltos por orden de SKU, así que las partes siempre suman exacto.
- **Obsequios (BR-04).** Un obsequio es una línea aparte con su cantidad, origen GIFT:<id> y precio 0. Su valor de referencia es cantidad × precio de lista. La política tributaria del país decide si ese valor paga impuesto (TAXED) o no (EXEMPT).

Descartar los demás órdenes quedó explícito en el ADR: aplicar escalas antes que combos descontaría dos veces las mismas unidades.

5.4 Puntos de redondeo

Solo se redondea por línea:

Punto	Dónde	Regla
RP-1	Precio unitario × cantidad	Exacto, porque los precios ya están en la escala de la moneda
RP-2	Cada componente de descuento de una línea (escala, asignación de combo)	Redondeado a la escala de la moneda. Un componente que redondea a cero no se emite
RP-3	Neto = bruto − componentes	Exacto

RP-4	Impuesto de una línea	Redondeado a la escala de la moneda
RP-5	Totales del pedido	Sumas de valores de línea ya redondeados, nunca redondeadas otra vez

Por qué por línea: la factura electrónica lleva el impuesto por línea, así que redondear por línea mantiene el pedido coherente con la factura que genera. Se descartó redondear una vez por grupo de tasa, porque los valores de línea no cuadrarían con los totales, y redondear solo el total final, porque esconde la deriva de centavos y no se puede auditar línea por línea.

Todo desglose cumple las identidades de BR-05, y todo golden test las verifica: subtotal bruto = Σ bruto pagado; Σ descuentos del pedido = Σ descuentos por escala de las líneas + Σ asignaciones de combo de las líneas; neto de línea = bruto − descuentos − asignaciones; base gravable = Σ neto pagado + Σ base gravable de obsequios; impuestos = Σ impuesto de línea; **total = Σ neto + impuestos**.

5.5 Los seis golden tests

Un caso de precios es una carpeta en src/test/resources/golden/ con request.json, snapshot.json y expected.json, marcada con @Tag("golden"). GoldenPricingTest revisa cada carpeta dos veces: contra su desglose esperado y contra las identidades anteriores. Todos los montos de abajo son pesos colombianos (COP, escala 0) tomados de los fixtures.

Prueba	**Caso del enunciado**	**Qué demuestra**
GT-01 gt01-two-tier-brackets	Descuento en dos tramos	25 gaseosas de 350 ml a 2,500, con tramos 10–19 → 5% y 20+ → 10%: 1,250 + 1,500 = 2,750 en una sola entrada TIER. Impuesto del 19% sobre 59,750 = 11,352.50 → 11,353. Total 71,103. Tramos marginales, redondeados una vez por línea
GT-02 gt02-combo-no-loose-discount	Combo cuyos productos no son un descuento suelto	21 gaseosas y 10 papas. El combo de 2 gaseosas y 1 papas a 5,000 se aplica 10 veces y consume 20 gaseosas y 10 papas: una sola entrada COMBO de 18,000. La gaseosa que sobra queda en el primer tramo, así que no aparece ninguna entrada TIER, aunque hay una regla de escala sembrada sobre el mismo SKU y 21 gaseosas solas alcanzarían el tramo del 10%. Total 60,622
GT-03 gt03-gift	Obsequio con su cantidad	25 papas y 3 aguas. La regla "un agua gratis por cada 10 snacks, repetible" agrega una línea de

		obsequio aparte de 2 aguas con origen GIFT:<id>. Las líneas pagadas no cambian. Colombia grava los obsequios. Total 53,175
GT-04 gt04-combo-with-taxes	Combo con impuestos	4 gaseosas y 2 papas. El combo se aplica dos veces: un descuento de 3,600, repartido en 2,647 y 953 por mayor residuo. Impuestos de 1,397 al 19% y 132 al 5%, así que 1,529 entran al total: 11,529
GT-05 gt05-credit-exceeded	Pedido a crédito que excede el cupo	Una cotización a crédito de 40 aguas y 60 gaseosas de 1.5 l suma 442,680 contra 400,000 disponibles: fits=false, faltante de 42,680. Corre por el caso de uso de cotización (GoldenQuoteTest#gt05), porque el cupo está fuera del motor puro. Su prueba compañera CreditLimitIT intenta confirmarla: 409 CREDIT_LIMIT_EXCEEDED, sin pedido y sin fila en el libro de crédito
GT-06 QuoteConfirmRegressionIT	Cotizar y confirmar dan el mismo total	Contra PostgreSQL y los datos semilla. Entre la cotización y la confirmación cambia el precio de las papas y se desactivan las palancas. El total confirmado es igual al cotizado, **y** una cotización nueva del mismo carrito da otro total, lo que demuestra que la prueba sí puede fallar. Un segundo caso muestra que la cotización sellada conserva su combo, su escala y su obsequio

En el mismo arnés corren más casos: las dos políticas tributarias de obsequios, los bordes de redondeo en .5, el mismo carrito en Colombia (escala 0) y en Ecuador (escala 2), y carritos sin palancas. TierPrecedenceTest demuestra que la regla por SKU le gana a la regla por familia.

6. El pedido idempotente (A2) paso a paso

6.1 La cotización sellada

POST /api/v1/quotes normaliza el carrito, resuelve la foto de los datos, corre el motor e inserta la cotización, sus líneas y sus componentes de descuento. Cada línea guarda el precio unitario y la tasa de impuesto vigentes en ese momento, además de la base gravable y el impuesto ya redondeados. Estas filas son de solo inserción: no existe ninguna sentencia UPDATE para las cotizaciones.

POST /api/v1/orders no escribe ningún monto. La fila del pedido guarda su número, su cotización, su cliente y su hora de confirmación. El desglose se arma desde las líneas de la cotización, así que cada cifra existe una sola vez, y el total confirmado es el cotizado por construcción. El motor nunca se vuelve a correr al confirmar: si se hiciera, un despliegue que cambiara el motor cambiaría todas las cotizaciones abiertas. El estado de la cotización se deriva, nunca se guarda: CONFIRMED si existe un pedido, si no EXPIRED después de expiresAt, y si no OPEN.

Alternativa descartada	Por qué
Copiar el desglose en el pedido	Dos copias pueden divergir
Recalcular al confirmar y comparar	Falla cada vez que cambia un precio o una palanca
Bloquear precios mientras haya cotizaciones abiertas	Bloquea las actualizaciones del catálogo

La contrapartida es explícita: un cambio de precio solo aplica a cotizaciones nuevas, y una cotización abierta conserva su precio hasta vencer, a los 15 minutos por defecto. Ese es el riesgo registrado en el ADR.

6.2 Una sola transacción

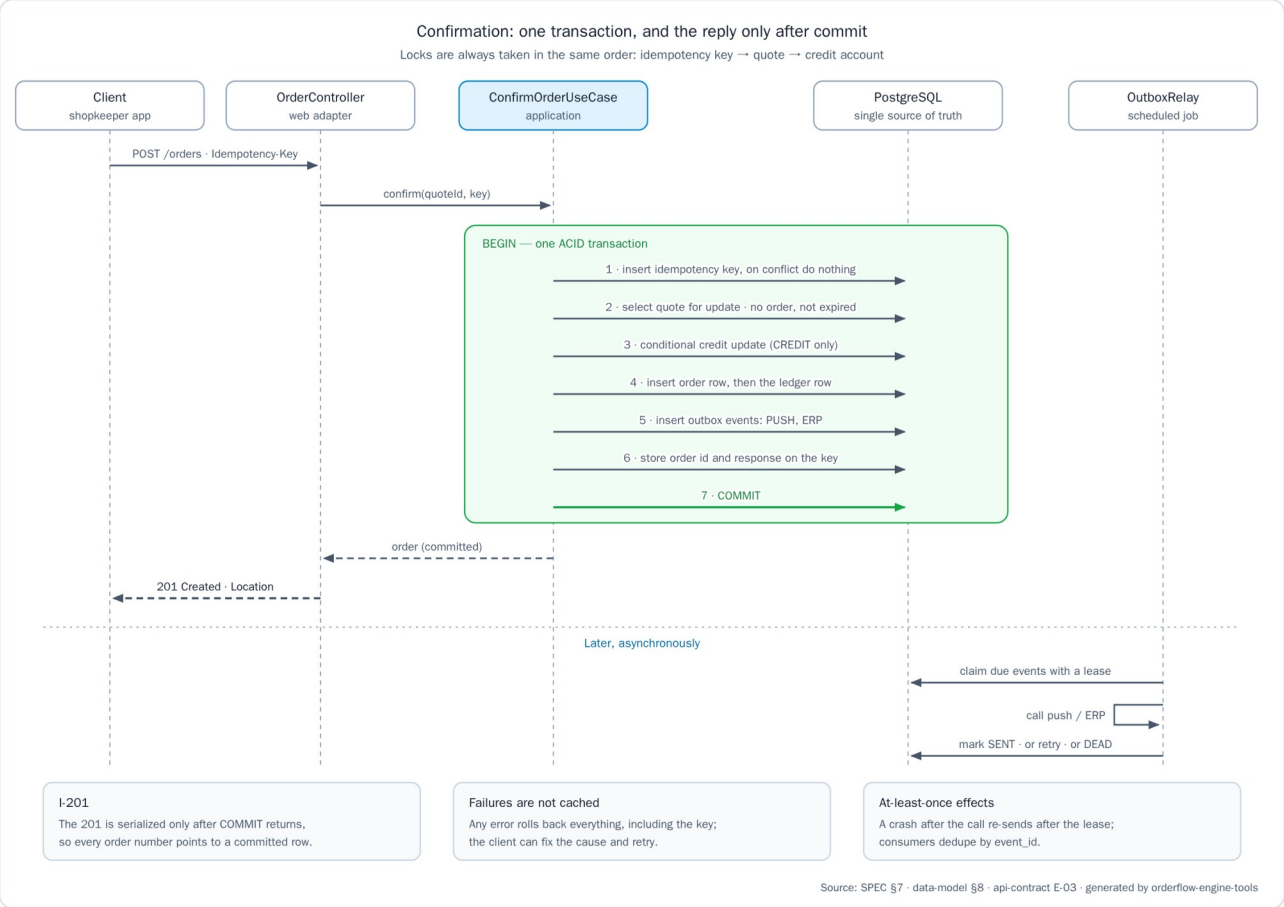


Figura 2. Confirmación: una transacción, y la respuesta solo después del commit

Una confirmación es una transacción ACID en PostgreSQL, en READ COMMITTED con bloqueos explícitos de fila y un lock_timeout. Los bloqueos se toman siempre en el mismo orden, clave de idempotencia → cotización → cuenta de crédito, así que las confirmaciones concurrentes no pueden caer en un deadlock.

Paso	Acción	Garantía
1	Insertar la clave de idempotencia con ON CONFLICT DO NOTHING y volver a leerla	Un solo dueño por clave. Un duplicado concurrente espera sobre la entrada única aún sin confirmar
2	Bloquear la cotización con SELECT … FOR UPDATE; verificar su dueño, que no tenga pedido y que no haya vencido	Una sola confirmación por cotización
3	Si es a crédito, consumir el cupo con un update condicional	El cupo nunca se consume por encima del límite
4	Insertar el pedido (único por cotización) y luego, si es a crédito, su fila en el libro	Ningún pedido sin su cotización, ningún segundo pedido para una misma cotización
5	Insertar un evento de outbox para push y otro para el ERP	Los efectos quedan registrados de forma atómica con el pedido
6	Guardar el cuerpo del 201 en la fila de idempotencia	Las repeticiones devuelven la misma respuesta
7	Hacer commit, y solo entonces serializar el 201	Invariante I-201

Las elecciones de tecnología salen de esta transacción (plan DEC-01 … DEC-12): Spring Boot sobre el stack servlet, porque un stack reactivo complica la única transacción bloqueante; un monolito modular, porque separar pedido, cupo, clave y outbox en servicios exigiría una saga; PostgreSQL, porque un almacén en memoria no puede demostrar atomicidad ni bloqueos de fila; JdbcClient con SQL explícito en lugar de JPA, que esconde los bloqueos y el orden de flush; y Testcontainers en lugar de H2, cuyo manejo de bloqueos y dialecto SQL son distintos. @Transactional solo aparece en los casos de uso de la aplicación, y una regla de ArchUnit lo hace cumplir.

6.3 La clave de idempotencia

Idempotency-Key es obligatoria en POST /api/v1/orders y debe cumplir ^[A-Za-z0-9_-]{8,64}$. El hash de la solicitud es SHA-256 de método, ruta, id del cliente y cuerpo JSON canónico. La semántica sigue el borrador del IETF "The Idempotency-Key HTTP Header Field":

Situación	Respuesta
Misma clave, mismo cuerpo, ya completada	La respuesta guardada, idéntica byte a byte, con Idempotent-Replayed: true
Misma clave, otro cuerpo	422 IDEMPOTENCY_KEY_REUSED
Misma clave mientras la primera solicitud sigue en curso	Espera hasta lock_timeout y repite la respuesta; 409 IDEMPOTENCY_KEY_IN_PROGRESS si primero se cumple el plazo
Clave ausente o mal formada	400 IDEMPOTENCY_KEY_MISSING o IDEMPOTENCY_KEY_INVALID
Una solicitud fallida (validación, cotización vencida, cupo excedido)	No se guarda: el rollback libera la clave, y el cliente puede reintentar después de corregir la causa
Una clave nueva sobre una	409 QUOTE_ALREADY_CONFIRMED, con el

cotización confirmada	número del pedido existente

Las claves se guardan 24 horas. Después de ese plazo, un reintento sigue recibiendo QUOTE_ALREADY_CONFIRMED con el número del pedido. El cliente web completa la garantía de su lado: una clave por cotización, creada en el primer toque de "Confirmar pedido", guardada en el navegador y reutilizada en cada reintento, con el botón deshabilitado mientras la solicitud está en curso.

6.4 El libro de crédito

Al cotizar, una cotización a crédito informa credit {fits, available, shortfall}, donde disponible = límite − consumido, y un total igual al disponible sí cabe. No se reserva nada. Al confirmar, dentro de la transacción:

1. UPDATE credit_accounts SET consumed = consumed + :total … WHERE … AND consumed + :total <= credit_limit.
2. Si no se actualiza ninguna fila, la respuesta es 409 CREDIT_LIMIT_EXCEEDED y el rollback no deja ni pedido ni fila en el libro.
3. Si no, se inserta una fila en el libro (order_id, amount) justo después del pedido. El libro es de solo inserción, con a lo sumo una fila por pedido.

La base de datos respalda al código: un CHECK mantiene consumed entre 0 y el límite aunque el código de la aplicación estuviera mal, y consumed = Σ ledger amounts se cumple siempre. CreditLimitIT y ConfirmOrderConcurrencyIT lo demuestran.

6.5 El outbox

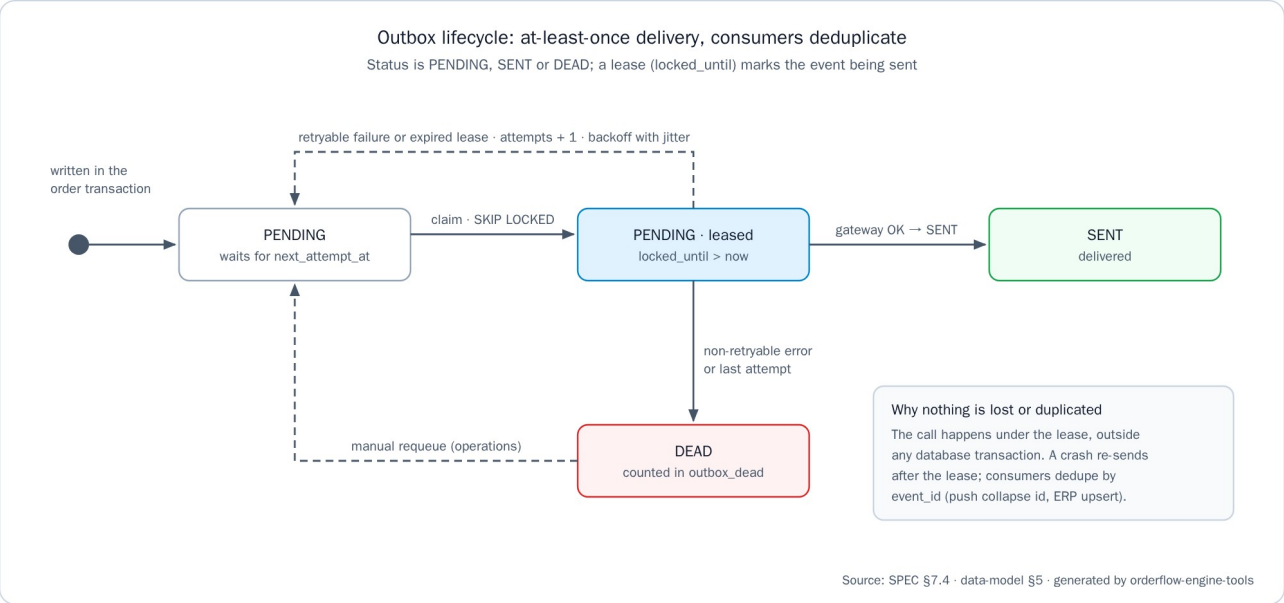


Figura 3. Ciclo de vida del outbox: entrega al menos una vez, los consumidores deduplican

Etapa	Comportamiento
Escritura	Un evento por destino (PUSH, ERP), escrito en la transacción del pedido. Su id es el event_id
Reclamo	Por defecto, el relay consulta cada segundo y toma en lease los eventos PENDING que ya vencieron con FOR UPDATE SKIP LOCKED, por 30 segundos, así que pueden correr varias instancias a

	la vez
Envío	Fuera de cualquier transacción de base de datos, pasando el event_id hacia abajo
Marca	Éxito → SENT. Una falla reintentable se reintenta con backoff exponencial y jitter completo, desde 2 segundos hasta 5 minutos. Después de 8 intentos, o de inmediato ante un error que reintentar no arregla, como un 4xx del ERP → DEAD, contado por el gauge outbox_dead y reencolado con el procedimiento del README de la API
Nunca se pierde	Si el servicio se detiene entre el envío y la marca, el lease vence y el evento se vuelve a enviar
Nunca se aplica dos veces	Los consumidores deduplican por event_id: push lo usa como collapse id, y el ERP hace upsert por número de pedido. Los gateways falsos del repositorio deduplican igual y se les puede ordenar que fallen
Sistema externo caído	El pedido sigue confirmado. El efecto espera en el outbox, outbox_pending muestra el atraso y la entrega se reanuda al recuperarse

Se descartó publicar después del commit porque una caída pierde el evento; el commit en dos fases, porque push y el ERP no pueden participar; y la captura de cambios (CDC), porque es demasiada infraestructura para este alcance. Las pruebas son OutboxWriteIT (exactamente un evento de push y uno de ERP por pedido) y OutboxRelayIT (recuperación tras una caída, vencimiento del lease con deduplicación, DEAD).

6.6 Modelo de consistencia y lo que sacrifica

Alcance	Modelo	Lo que se sacrifica
Pedido, cupo, clave de idempotencia, eventos del outbox	Fuerte: una transacción ACID en una sola base de datos	Disponibilidad mientras la base de datos está caída. Las confirmaciones de un mismo cliente se serializan sobre su fila de crédito. El camino de escritura depende de una sola base de datos
Push y ERP	Eventual: entrega al menos una vez, deduplicada por los consumidores	Van detrás de la confirmación por el intervalo de consulta más los reintentos

6.7 Las "tres releases": hipótesis de causa raíz y la prueba que cierra cada una

El enunciado cuenta pedidos que se confirmaron, recibieron número y después no aparecían. La causa nunca se dijo, así que se listó cada causa raíz plausible (SPEC §7.6), se le dio una regla de diseño y se cerró con una prueba que falla si la regla se rompe:

Causa raíz plausible	Respuesta de diseño	Prueba que la cierra
La respuesta se envió	El 201 se serializa	ConfirmOrderCommitOrderingIT: cuando

antes del commit	solo después del commit (I-201)	se escribe la respuesta, no hay transacción abierta y otra conexión ya lee el pedido. Una falla que lanza el propio commit, mediante un trigger de restricción diferido, nunca se responde con un 201, y un reintento posterior confirma una sola vez
@Transactional se saltó por autoinvocación	El límite de la transacción es un bean de caso de uso aparte, y ArchUnit lo hace cumplir	TransactionBoundaryArchTest, ConfirmOrderFailureInjectionIT
El historial se leía de una réplica o del ERP	El detalle del pedido y el historial leen las tablas primarias de forma síncrona	OrderReadYourWritesIT: todo pedido con 201 se puede leer de inmediato por id y en el historial
El pedido se escribía de forma asíncrona después de responder	La fila del pedido se escribe en la transacción de la solicitud; solo los efectos son asíncronos	ConfirmOrderFailureInjectionIT: un trigger de base de datos hace fallar una de cinco escrituras antes del commit (update del cupo, pedido, libro, outbox, clave). No persiste nada y la clave se libera; el reintento con la misma clave crea exactamente un pedido. Cuando la respuesta se pierde después del commit, el reintento devuelve el mismo número de pedido, byte a byte, como repetición
Una actualización perdida o un duplicado por doble toque	Clave de idempotencia única, una sola orden por cotización, update condicional del cupo	ConfirmOrderConcurrencyIT: 6 solicitudes concurrentes con una misma clave, enviadas mientras la fila de crédito está retenida, reciben todas un 201 con el mismo cuerpo, solo una no es repetición, y queda un pedido, una fila en el libro y dos eventos de outbox. Dos confirmaciones que juntas exceden el cupo: exactamente una tiene éxito

El cliente web cierra el mismo defecto de su lado: no guarda copia de los pedidos, y Mis pedidos lee el historial en cada visita, así que ninguna caché del cliente puede esconder un pedido confirmado (AC-52, demostrado por history-read-your-writes.spec.ts y el flujo de extremo a extremo).

7. El ADR (A3)

El enunciado pide un ADR de una página como máximo, así que la entrega tiene exactamente uno: orderflow-engine-api/docs/adr/0001-order-pricing-and-confirmation.md, "Cotizaciones selladas y confirmación en una sola transacción", con su par en español.

Usa la tabla de encabezado de MADR 4.0.0 (estado, fecha, decisores, confianza) y las cuatro secciones que exige A3.

Sección	Contenido en breve
Decisiones y alternativas descartadas	Cotización sellada y referenciada por el pedido (ni copiar ni recalcular); BigDecimal redondeado por línea, combos antes que escalas (ni unidades menores en long, ni redondear el total, ni escalas primero); una transacción en PostgreSQL con bloqueos clave → cotización → cupo y el 201 después del commit (ni memoria, ni escritura asíncrona o saga, ni claves en una caché); outbox transaccional con lease y backoff (ni publicar después del commit, ni commit en dos fases); monolito hexagonal, SQL explícito, repositorios separados (ni microservicios, ni JPA, ni monorepo)
No implementado, a propósito	Autenticación, clientes reales de push y ERP, y lo que no se pidió
Qué revisaría en un code review	Ni float ni double; compareTo para el dinero; ningún UPDATE sobre cotizaciones; @Transactional solo en casos de uso; orden de bloqueos; 201 después del commit; ninguna falla guardada; relay fuera de transacciones; cada palanca con un golden test que se vio fallar; texto de usuario solo en i18n
Riesgo y mitigación	Una cotización sellada respeta un precio equivocado hasta vencer. Mitigación: un TTL de 15 minutos por país y, después, la revocación (OPEN → REVOKED, 409). Revisar si un incidente afecta más de 100 cotizaciones o si el TTL pasa de una hora

El límite de una página se hace cumplir dos veces: check-docs falla por encima de 500 palabras por idioma, y la generación en Word y PDF (Arial 12, A4, márgenes de 2 cm) falla si pasa de una página. El ADR entró como borrador en T-00 (docs(api): add ADR-0001 draft (T-00)) y se aceptó en T-16 (docs: accept ADR and record evidence (T-16)). Todas las demás decisiones, con sus alternativas descartadas, viven en orderflow-engine-docs/spec/plan.md, así que el ADR se mantiene en una página.

8. El cliente web y el stack completo

8.1 Del plus de una hora al flujo completo

El plus del enunciado es una pantalla de resumen del pedido en una hora como máximo. Esa pantalla se construyó primero y sola, para poder medir su tiempo: 68 minutos, que ya superan la hora cuando se cuentan sus pruebas y su contenedor. El 2026-10-15 el alcance del front se amplió a propósito, como decisión de product owner registrada en SPEC §1 ("Deviation"), al flujo de dominio que describe el enunciado:

Ruta	Pantalla	Operaciones de la API
/cart, también /	Carrito: grid de productos, cantidades, contado o crédito	E-07 (lista de productos), E-01 (cotización)
/quotes/:quoteId	Resumen del pedido, la pantalla del enunciado	E-02, E-03 (confirmación)
/orders/:orderId	El pedido confirmado	E-04

/orders	Mis pedidos	E-05

La ampliación quedó contenida: empezó solo después de integrar T-16, no agregó ninguna regla de negocio, nunca calcula dinero (cada monto es una cadena de la API) y sumó una sola operación de la API, de solo lectura, GET /api/v1/products (E-07). Un selector de tienda envía un tenant y un cliente sembrados como encabezados, en lugar de un inicio de sesión (A-06). El español es el idioma por defecto y el inglés está completo; el selector ES/EN cambia el idioma en tiempo de ejecución y el navegador recuerda la elección.

8.2 Prevención a nivel de componente

Cada defecto que nombra el enunciado tiene un componente dueño, así que una pantalla nueva hereda la prevención en lugar de resolverla otra vez:

Defecto	Componente	Regla
Texto cortado	ui-text	La única forma en que una pantalla imprime texto. Siempre hace salto de línea, nunca usa puntos suspensivos ni límite de líneas, y un monto nunca se parte
Controles tapados por la barra del sistema o el teclado	ui-action-bar	Fija abajo, con relleno según las zonas seguras, y se eleva sobre el teclado siguiendo visualViewport; cada control mide al menos 44 × 44 px
Imágenes de un grid escaladas distinto	ui-product-image	Una sola relación de aspecto con object-fit: contain, la única forma de dibujar una imagen de producto, con una imagen de reemplazo cuando falta una

Las specs miden el layout real en Chromium headless a 768 × 1024 y 375 × 812, con nombres de productos y tiendas muy largos, porque jsdom no tiene motor de layout y ahí una verificación de salto de línea pasa con cualquier layout (trampa TR-11 del HANDOFF). Además, un script hace fallar la compilación si alguna plantilla contiene texto literal.

8.3 El stack completo y el flujo de extremo a extremo

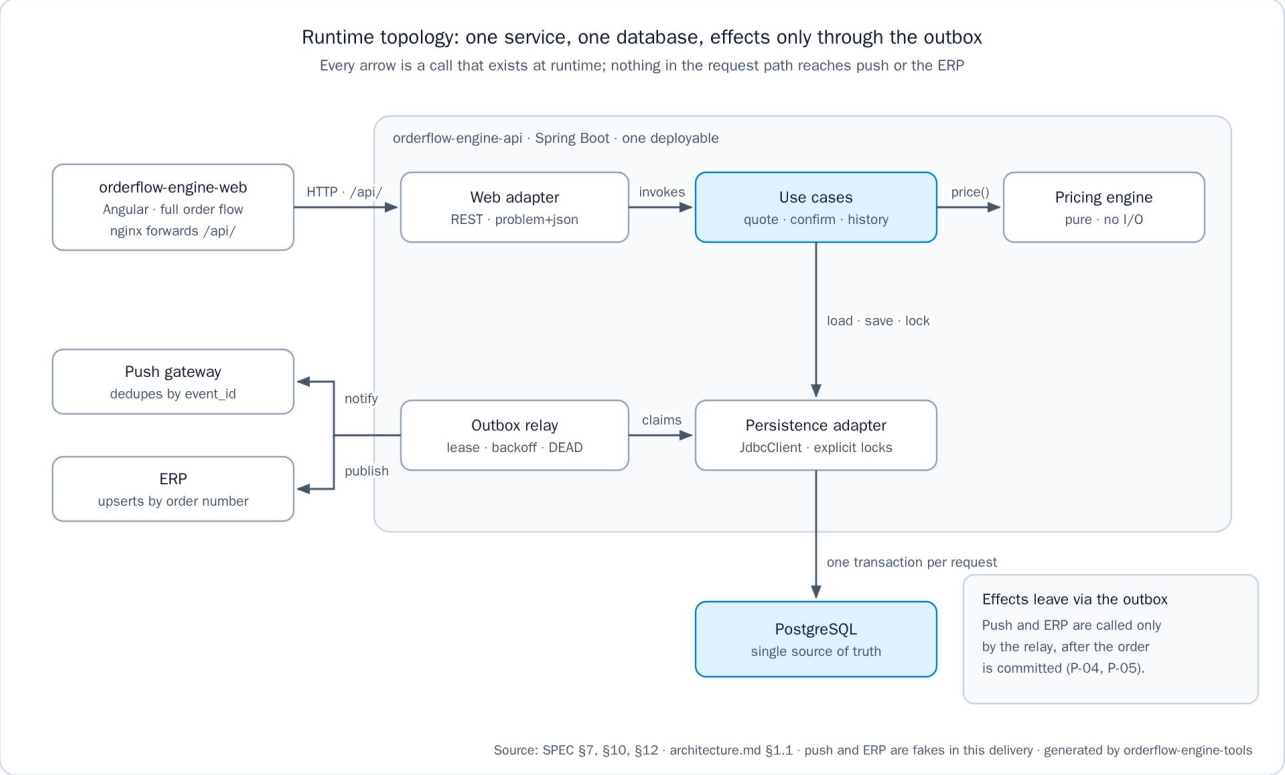


Figura 4. Topología en ejecución: un servicio, una base de datos, efectos solo a través del outbox

El nginx del cliente web reenvía /api/ a la API, así que el navegador ve un solo origen y la API no necesita configuración de CORS. El repositorio orderflow-engine-infra levanta PostgreSQL, la API y el cliente web con un solo docker compose up --build, cada servicio con restart: unless-stopped. Su test-all ejecuta, en orden, la verificación de la API, las specs web en su contenedor, un flujo de extremo a extremo con Playwright contra el stack completo (carrito → cotización → confirmación → pedido → Mis pedidos, en español y en inglés, con su propio proyecto de compose y sus propios puertos), la verificación de documentación y la de integridad.

9. Países y datos semilla

Países, monedas, tenants, catálogos, palancas, clientes y cuentas de crédito son datos, escritos como changesets de Liquibase en orderflow-engine-api/src/main/resources/db/changelog/. Cada changeset tiene un rollback, cada changelog fija un logicalFilePath, y LiquibaseRollbackIT demuestra que todos vuelven a un esquema vacío y se aplican de nuevo. Los datos semilla son de demostración, elegidos para cubrir las dos escalas y las dos políticas tributarias de obsequios; no describen la legislación tributaria real.

Tenant	Política del país	Catálogo y palancas	Clientes y cupo
co-alpha	COP escala 0, HALF_UP, obsequios TAXED, TTL 15 min	Gaseosas, agua y papas; impuesto del 19% y del 5%. Escala en la gaseosa de 350 ml (10 → 5%, 20 → 10%); combo de 2 gaseosas y 1 papas a	Un cliente con cupo de 500,000 y otro que paga de contado

		5,000; un agua gratis por cada 10 snacks	
ec-alpha	USD escala 2, HALF_UP, obsequios EXEMPT, TTL 15 min	Impuesto del 15%; escala por familia en gaseosas (12 → 4%, 24 → 8%); un combo a 3.99; un obsequio desde un subtotal bruto de 50.00	Una cuenta de crédito con cupo de 1,500.00
gt-alpha	GTQ escala 2, HALF_UP, obsequios EXEMPT, TTL 15 min	Impuesto del 12%; escala en la gaseosa de 350 ml (12 → 5%, 24 → 10%); un agua gratis por cada 12 gaseosas	Un cupo de 5,000.00 y una tienda que paga de contado
pe-alpha	PEN escala 2, HALF_UP, obsequios TAXED, TTL 15 min	Impuesto del 18%; combo de 2 gaseosas y 1 papas a 6.00; un agua gratis desde un subtotal bruto de 100.00	Un cupo de 2,000.00 y una tienda que paga de contado

Los datos semilla crecieron en tres changelogs. 2026-10-15-006-seed-co-and-ec.yaml sembró Colombia y Ecuador en T-08. 2026-10-16-001-seed-names-in-spanish.yaml renombró productos y clientes al idioma de los tenants en T-19, porque un changeset ya aplicado nunca se edita. 2026-10-16-002-seed-gt-and-pe.yaml agregó Guatemala y Perú en T-20 sin tocar Java, lo que en sí mismo demuestra que ninguna moneda, escala, tasa o regla está quemada en el código (P-07). ProductContractIT lista GTQ y PEN con escala 2, y QuoteLeversIT cotiza un carrito de Guatemala (escala y obsequio sin impuesto) y uno de Perú (combo y obsequio gravado) cuyos totales se calcularon a mano. Los números de pedido salen de una secuencia por país, como CO-000042. El selector de tienda web lista las ocho tiendas sembradas, dos por tenant.

La administración del catálogo queda fuera del alcance por diseño (SPEC §1): una tienda, un producto o una palanca nuevos son un changeset nuevo.

10. Cómo se demostró la calidad

Cada tarea se construye con el mismo ciclo (TASKS, "Task cycle"): escribir la prueba que falla y verla fallar por la razón esperada; implementar lo mínimo para que pase; **demostrar que la prueba muerde**, rompiendo una vez la protección (invertir una regla, quitar un bloqueo, saltarse un redondeo) y viendo fallar la prueba; correr la verificación completa; y abrir el pull request con la salida de la verificación. Ejemplos registrados de una protección rota a propósito: cada regla de pureza del dominio falló con una clase de prueba, el umbral de JaCoCo falló con una clase sin cubrir (HANDOFF al cierre del andamiaje), y una spec web que fallaba hizo fallar también test-all en PowerShell (el pull request que sumó el cliente web al stack completo).

Práctica	Cómo se aplicó	Evidencia (HANDOFF §3)
Pruebas unitarias y golden tests	JUnit 5 y AssertJ; fixtures JSON para desgloses completos con reloj fijo e ids fijos	333 pruebas unitarias en verde, entre ellas los seis golden tests
Integración sobre	Testcontainers con PostgreSQL	308 pruebas de integración en

una base de datos real	17, nunca H2	verde
Concurrencia e inyección de fallas	Solicitudes concurrentes con una clave, una fila de crédito retenida, triggers que hacen fallar una escritura, un trigger diferido que hace fallar el commit, un filtro que pierde la respuesta después del commit	ConfirmOrderConcurrencyIT, ConfirmOrderFailureInjectionIT, ConfirmOrderCommitOrderingIT
Cobertura	Umbral de JaCoCo de al menos 90% de líneas y ramas en pricing, que hace fallar ./mvnw verify	100% de líneas y ramas en pricing.domain y pricing.domain.lever; 99.2% de líneas y 98.0% de ramas en todo el servicio
Pruebas de mutación	PIT en el perfil opcional ./mvnw -Pmutation verify, umbral de 80%	185 de 185 mutaciones detectadas (100%)
Reglas de arquitectura	ArchUnit: dominio puro, ni float ni double, nada de java.util.Currency en el dominio, @Transactional solo en casos de uso	DomainPurityArchTest, TransactionBoundaryArchTest
Contrato e i18n	MockMvc sobre códigos de estado, forma de problem+json y textos localizados; cada código de error tiene mensaje en es y en en	ProblemDetailContractIT, LocalizedErrorIT, ErrorCatalogI18nTest
Specs web	Chromium headless dentro de la etapa de pruebas del Dockerfile web; cada spec responde con un backend simulado y verifica la solicitud exacta	209 specs en 15 archivos
Extremo a extremo	Playwright contra el stack completo, en español y en inglés	El flujo pasa 2 de 2
Verificación sobre exportación limpia	Desde la fase de andamiaje, cada pull request de la API se verificó sobre una exportación limpia de su propio commit (git archive de la cabeza de la rama y luego ./mvnw verify), así que cada commit en dev compila por sí solo; el único pull request de solo documentación de la fase núcleo se verificó con ./mvnw verify sobre su commit. Cada pull request web corrió docker build --no-cache-filter test --target test sobre una exportación limpia, y cada pull	HANDOFF §9 … §11

	request de infra y tools corrió sus propias verificaciones	
Verificación del espacio de trabajo	Desde la fase de andamiaje, test-all pasó antes de cada cierre de fase. Antes de cerrar la fase web pasó sobre clones nuevos desde GitHub de dev de cada repositorio de código y de main del repositorio de documentación, el 2026-10-17	test-all: OK (API verify, web tests, end-to-end, documentation check, integrity check)

La batería de pruebas creció con el trabajo: 3 pruebas al cierre del andamiaje (las dos reglas de pureza del dominio y la verificación de salud), 310 unitarias y 287 de integración al cierre del núcleo, y 333 unitarias y 308 de integración al cierre de la fase web, según registran las versiones sucesivas del HANDOFF.

11. El historial de git

11.1 Repositorios y ramas

Repositorio	Rol	Ramas permanentes
orderflow-engine-api	**El entregable**: servicio, pruebas, README, ADR	main, dev
orderflow-engine-web	El cliente Angular	main, dev
orderflow-engine-infra	Compose del stack completo y test-all, con la suite de extremo a extremo	main, dev
orderflow-engine-tools	Verificaciones y generadores para documentos y repositorios	main, dev
orderflow-engine-app	Reservado para una futura app móvil; no se construyó	main, dev
orderflow-engine-docs	Especificación, gobierno, handoff, diagramas, entregables	Solo main

Los seis repositorios se crearon el 2026-10-14. La regla de oro (orderflow-engine-docs/governance/golden-rule.md) dice que nada llega a main si no es por un pull request. En un repositorio de código, las ramas de tarea se integran a dev con squash, y main recibe dev solo cuando se cierra una fase, con un merge commit, así que cada commit de tarea sigue alcanzable desde main. El repositorio de documentación solo tiene main, porque guarda documentación viva. GitHub rechaza los rulesets de ramas en repositorios privados de un plan gratuito (HTTP 403), así que las reglas de protección se cumplieron a mano mientras los repositorios son privados (trampa TR-06 del HANDOFF).

11.2 Pull requests por fase

Según gh pr list --state merged el 2026-10-17, se integraron 82 pull requests: 31 en la API, 10 en el cliente web, 12 en infra, 17 en docs, 10 en tools y 2 en app.

Fase	Pull requests de tarea	Pull requests de cierre de

		fase
1. Preparación (T-00)	21	5
2. Andamiaje (T-01)	9	2
3. Núcleo (T-02 … T-16, T-18)	20	3
4. Web (T-17, T-19, T-20)	18	4

La fila web cuenta lo que gh listaba el 2026-10-17. El HANDOFF lista un pull request de tarea número 19 para esa fase: su propio registro del cierre de fase, en el repositorio de documentación.

El HANDOFF lista cada pull request con su número y su asunto en §8 … §11. Los pull requests de tarea de la fase núcleo en la API, en orden de integración, se leen como la historia de la construcción:

```
feat(pricing): add money value object (T-02)
feat(pricing): add pricing pipeline and goldens (T-03)
feat(db): add normalized schema and seeds (T-08)
feat(api): add localized problem details (T-09)
feat(outbox): relay order events reliably (T-14)
feat(outbox): purge expired data by retention (T-15)
feat(quoting): add sealed quotes (T-10)
feat(ordering): confirm orders atomically (T-11)
test(ordering): prove confirmation invariants (T-12)
feat(ordering): add order history endpoints (T-13)
feat(pricing): apply marginal tier discounts (T-04)
feat(pricing): apply combos before tiers (T-05)
feat(pricing): add gifts with tax policy (T-06)
test(pricing): cover rounding and countries (T-07)
feat(quoting): price quotes with seeded levers (T-18)
docs: accept ADR and record evidence (T-16)
```

11.3 Commits convencionales

Cada commit es una sola línea que sigue Conventional Commits 1.0.0 y termina con su tarea: <type>(<scope>): <imperative summary> (T-NN), de 60 caracteres como máximo y en inglés (orderflow-engine-docs/governance/commit-conventions.md). Los pull requests de tarea se integran con squash, así que el título del pull request es el commit, y la razón y la salida de la verificación quedan en el cuerpo del pull request (tarea, criterios cubiertos, verificación, notas). Un cierre de fase conserva su etiqueta de tarea, como chore: close core phase (T-16). check-docs falla si una línea de commit planeada en TASKS rompe estas reglas.

12. Qué no se hizo y por qué

Elemento	Por qué
Autenticación	Fuera del alcance. Tenant y cliente llegan en encabezados, y un gateway los inyectaría
Captura de pagos, cancelación	No se pidieron

y liberación de cupo	
Persistencia del carrito en el servidor	No se pidió. El carrito viaja en la solicitud de cotización, y el cliente web lo guarda en el navegador
Revocación de cotizaciones	No se pidió. Es el siguiente paso de la mitigación del riesgo del ADR
Administración de catálogo y palancas	No se pidió. Las migraciones siembran los datos. La única operación de catálogo es la lista de productos de solo lectura (E-07)
Categorías tributarias e historial de precios	No se pidieron. Cada SKU lleva su tasa, y cada línea de cotización guarda el precio y la tasa vigentes al cotizar
Clientes reales de push y ERP	Gateways falsos detrás de puertos demuestran las garantías de entrega
Archivo OpenAPI y despliegue	No se pidieron. El contrato es orderflow-engine-docs/spec/api-contract.md
Configuración de CORS	No hace falta: el nginx del cliente web reenvía /api/, así que el navegador ve un solo origen
SonarQube o SonarCloud, un pipeline de CI, pruebas de mutación en CI	Necesitan un servidor o configuración de CI. El proyecto ya produce lo que cada uno consume (el reporte XML de JaCoCo, los comandos)
Un job de migración aparte con roles de mínimo privilegio, un repositorio de esquema aparte	Liquibase corre al arrancar con la credencial del dueño, y los changelogs viven con su único dueño; cada changelog tiene un logicalFilePath, así que puede moverse sin volver a aplicarse
Catálogo en varios idiomas	Los nombres de producto son datos del tenant en su idioma; el selector ES/EN traduce la interfaz
App móvil	orderflow-engine-app está reservado y vacío

Cada elemento que un equipo podría adoptar después tiene un camino de adopción escrito, con el estado actual, el cambio que requiere y su disparador, en orderflow-engine-docs/spec/plan.md §3.

13. Tiempo invertido

Los tiempos medidos salen de la tabla "Tiempo invertido" del README de la API:

Parte	**Tiempo medido**
La prueba (A1, A2, A3), incluida la especificación	Tres días de trabajo, del 2026-10-13 al 2026-10-15
La pantalla opcional del enunciado: el resumen del pedido	68 min
Su contenedor, el cliente web en el stack completo y el paso web de test-all	55 min
La ampliación más allá del enunciado: la lista de productos (E-07), el carrito, la confirmación idempotente, el pedido, el selector de tienda y Mis pedidos	3 h 1 min

Los tiempos web son tiempo de trabajo por parte, construyéndola y corrigiéndola, tomados de marcas de tiempo registradas en cada paso; no incluyen el tiempo de

revisión. Las partes se construyeron entre el 2026-10-15 22:05 y el 2026-10-16 06:15 (UTC-5), varias de ellas en paralelo. El enunciado da cinco días; la última fase se cerró el 2026-10-17.

14. Cómo verificarlo uno mismo en 5 comandos

Desde una carpeta vacía, con Git, la CLI de GitHub, JDK 21, Node 18 o posterior y Docker corriendo (mientras los repositorios sean privados, hace falta acceso de lectura):

```
for r in api web infra docs tools app; do gh repo clone ariel5253/orderflow-engine-$r; done
(cd orderflow-engine-api && ./mvnw verify)
(cd orderflow-engine-api && ./mvnw -Pmutation verify)
bash orderflow-engine-infra/scripts/test-all.sh
(cd orderflow-engine-infra && docker compose up --build)
```

#	Comando	Qué demuestra
1	Clonar los seis repositorios uno al lado del otro	La disposición del espacio de trabajo que esperan test-all y el compose del stack completo
2	./mvnw verify en orderflow-engine-api	El comando único del enunciado: pruebas unitarias, golden, de contrato, de arquitectura y de integración contra PostgreSQL, y el umbral de cobertura
3	./mvnw -Pmutation verify en orderflow-engine-api	PIT sobre el dominio de precios, umbral de 80%
4	bash orderflow-engine-infra/scripts/test-all.sh	Todo: termina con test-all: OK (API verify, web tests, end-to-end, documentation check, integrity check)
5	docker compose up --build en orderflow-engine-infra	El stack en marcha: el cliente web en http://localhost:8081 y la API en http://localhost:8080

Con el stack corriendo, elige la tienda "Tienda Norte", agrega 25 unidades de "Gaseosa cola 350 ml", elige "Crédito" y presiona "Cotizar": el resumen muestra el golden test 1, un descuento por escala de 2,750, 11,353 de impuesto y un total de 71,103 COP. Presiona "Confirmar pedido", y el pedido aparece primero en "Mis pedidos" con el mismo total. En Windows, pwsh -File orderflow-engine-infra/scripts/test-all.ps1 reemplaza el paso 4.